

Unsupervised Anomaly Detection in Periodic Time Series

A Deep Learning Approach for ECG Arrhythmia Identification

Research Progress Presentation

A.K.S.S. Wimalasena
Supervisor: Ms. Sachini Karunaratne

Department of Mathematics
University of Ruhuna

April 2026

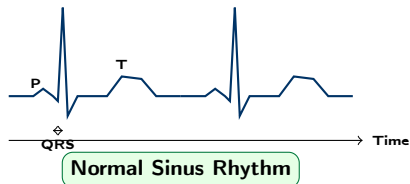
Outline

- 1 Introduction & Motivation
- 2 Literature Review
- 3 Methodology
- 4 Data Pipeline
- 5 Model Architecture
- 6 Mathematical Foundations
- 7 Implementation
- 8 Current Progress
- 9 Next Steps & Timeline
- 10 Conclusion

The Clinical Challenge

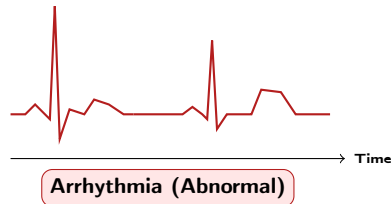
Cardiovascular Diseases (CVDs)

- Leading cause of death globally — 17.9 million/year (WHO)
- Many CVDs manifest as **arrhythmias** in ECG signals
- Early and accurate detection is **critical** for timely intervention



Electrocardiogram (ECG)

- Non-invasive recording of heart's electrical activity
- Normal sinus rhythm: **periodic, predictable** waveform
- Arrhythmias: disruptions in periodicity, morphology, or timing



Core Challenge

How can we automatically detect anomalies in periodic ECG signals **without requiring labeled arrhythmia examples** for training?

Key Challenges:

- 1 **Label Scarcity** — Abundant normal ECG data, but rare and diverse arrhythmias
- 2 **Temporal Complexity** — Nonlinear variations in biological periodic signals
- 3 **Generalization** — Must detect *unseen* anomaly types

Our Hypothesis:

Reconstruction-Based Detection

If a model learns to reconstruct only **normal** ECG patterns, then **abnormal** signals will produce high reconstruction errors, enabling unsupervised anomaly detection.

Gaps in Existing Work:

- ✗ Most methods require **labeled** arrhythmia data (supervised)
- ✗ Traditional methods treat data as disconnected points
- ✗ Cannot detect **unknown** arrhythmia types
- ✗ Limited end-to-end systems for clinical use

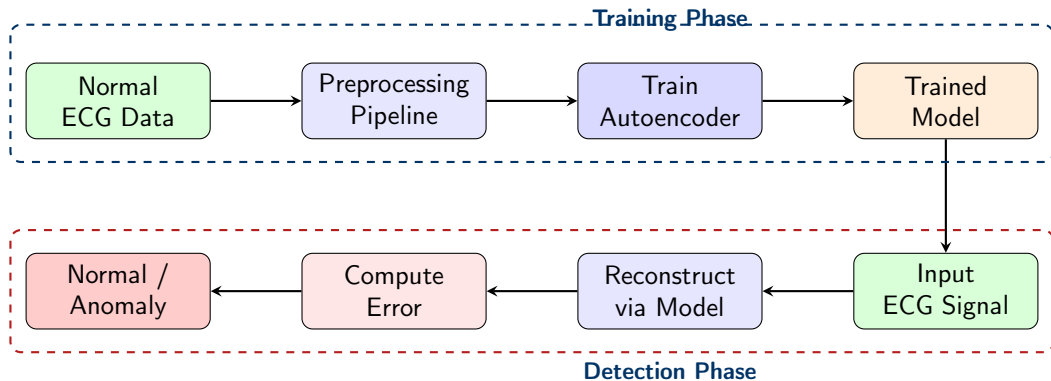
Our Contributions:

- ✓ **Unsupervised** approach — no arrhythmia labels needed
- ✓ Detects **any** deviation from normal patterns
- ✓ Complete system with **interactive web UI**
- ✓ Experiment tracking with **MLflow**

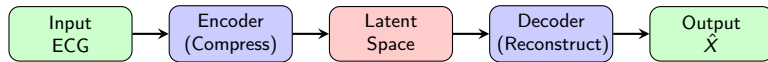
Table: Comparison of Approaches for ECG Anomaly Detection

Approach	Method	Supervision	Limitation	Generalize
Traditional	Threshold rules	None	Manual, rigid	No
Classical ML	SVM, k-NN	Supervised	Needs labels	No
RNN/LSTM	Sequential	Supervised	Expensive, labels	No
LSTM-AE	Seq. reconstruction	Unsupervised	Slow training	Partial

Overall System Architecture



Autoencoder for Anomaly Detection



Bottleneck forces learning of
essential normal patterns

Normal: Low error
 $\|X - \hat{X}\|^2 \approx 0$

Anomaly: High error
 $\|X - \hat{X}\|^2 \gg 0$

Key Insight

Normal ECG signals lie on a **low-dimensional manifold** $\mathcal{M}$ in high-dimensional signal space. The autoencoder learns to map to/from $\mathcal{M}$.

Normal Signal $X \in \mathcal{M}$

$$\hat{X} = \psi(\phi(X)), \quad \|X - \hat{X}\|_2^2 \approx 0 \quad (1)$$

Well reconstructed $\Rightarrow$ low anomaly score

Anomalous Signal $X' \notin \mathcal{M}$

$$\hat{X}' = \psi(\phi(X')), \quad \|X' - \hat{X}'\|_2^2 \gg 0 \quad (2)$$

Poorly reconstructed $\Rightarrow$ high anomaly score

where ϕ is the encoder and ψ is the decoder

Training: Normal Manifold

MIT-BIH Normal Sinus Rhythm DB

- 18 healthy subjects
- Sampling: 128 Hz
- Long-term ECG recordings
- Used for **training only**

Testing: Anomaly Detection

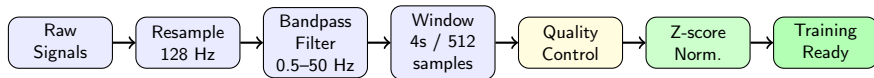
MIT-BIH Arrhythmia Database

- 48 subjects with arrhythmias
- Sampling: 360 Hz
- 15 annotated arrhythmia types
- Beat-level labels available

Design Principle

Model is trained on **only normal** data. Arrhythmia data is reserved **exclusively** for evaluation.

Preprocessing Pipeline



Signal Processing:

- Polyphase anti-aliasing for resampling
- Butterworth bandpass (0.5–50 Hz)
- Removes baseline wander & high-freq noise

Quality & Normalization:

- Quality control removes artifacts
- Z-score normalization per window
- Subject-wise split prevents leakage

Parameter	Value
Sampling Rate	128 Hz
Window Size	4 seconds
Samples/Window	512
Total Windows	~90,000
High-Quality	~72,000
Data Split	70/15/15%
Signal Shape	(1, 512)

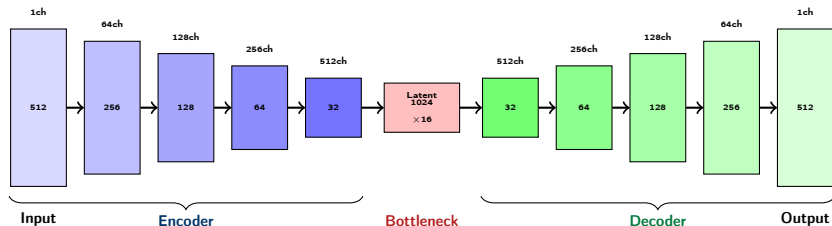
Data Splitting Strategy:

- **Subject-wise** partitioning
- Training: subjects 1–12
- Validation: subjects 13–15
- Test: subjects 16–18
- Prevents **data leakage**

Quality Control

Automated artifact detection retains ~80% of windows, ensuring clean training data.

1D Convolutional Autoencoder



Compression ratio: $512 \rightarrow 16$ temporal positions ($32\times$ compression)

Total parameters: $\sim 9.75\text{M}$ (symmetric encoder-decoder)

1D Convolutional Autoencoder

Table: Encoder

Layer	Shape In	Shape Out	Params
Conv1+BN+ReLU	(1,512)	(64,256)	512
Conv2+BN+ReLU	(64,256)	(128,128)	57K
Conv3+BN+ReLU	(128,128)	(256,64)	230K
Conv4+BN+ReLU	(256,64)	(512,32)	918K
Conv5+BN+ReLU	(512,32)	(1024,16)	3.67M
Encoder Total			4.88M

Table: Decoder (Transposed Conv)

Layer	Shape In	Shape Out	Params
TrConv1+BN+ReLU	(1024,16)	(512,32)	3.67M
TrConv2+BN+ReLU	(512,32)	(256,64)	918K
TrConv3+BN+ReLU	(256,64)	(128,128)	230K
TrConv4+BN+ReLU	(128,128)	(64,256)	57K
TrConv5+Sig	(64,256)	(1,512)	449
Decoder Total			4.88M

- **Kernel size: 7, Stride: 2, Dropout: 0.2**
- **Batch Normalization** after each convolution
- Final activation: **Sigmoid** (output $\in [0, 1]$)

Loss Function & Optimization

Training Loss (Mean Squared Error):

$$\mathcal{L}_{MSE} = \frac{1}{T} \sum_{t=1}^T \left(X[t] - \hat{X}[t] \right)^2 \quad (3)$$

Anomaly Score (Mean Absolute Error):

$$S(X) = \frac{1}{T} \sum_{t=1}^T \left| X[t] - \hat{X}[t] \right| \quad (4)$$

Optimization (Adam optimizer):

$$W_{t+1} = W_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad \text{with } \eta = 0.001 \quad (5)$$

- MSE for gradient-based optimization (smooth gradients)
- MAE for anomaly scoring (robust to outlier values)

Anomaly Detection: Thresholding

Step 1: Compute reconstruction $\hat{X} = \psi(\phi(X))$

Step 2: Calculate anomaly score $S(X) = \text{MAE}(X, \hat{X})$

Step 3: Determine threshold from validation set:

$$\tau = \mu_{val} + k \cdot \sigma_{val}, \quad k \in \{1, 2, 3\} \quad (6)$$

Step 4: Classification:

$$\text{Class}(X) = \begin{cases} \text{ANOMALY} & \text{if } S(X) > \tau \\ \text{NORMAL} & \text{if } S(X) \leq \tau \end{cases} \quad (7)$$

Threshold Selection

The parameter k controls the **sensitivity–specificity trade-off**. Higher $k \Rightarrow$ fewer false positives but potentially more missed anomalies.

Technology Stack

Core Framework:

- **Python 3.14** — Primary language
- **PyTorch** — Deep learning framework
- **NumPy / SciPy** — Signal processing
- **WFDB** — PhysioNet data loading

Experimentation:

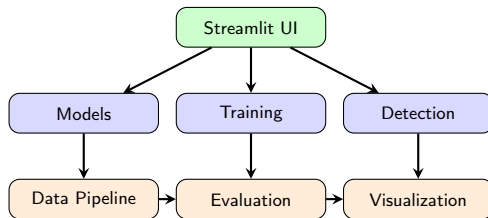
- **Jupyter Notebooks** — Interactive experiments
- **MLflow** — Experiment tracking
- **YAML configs** — Hyperparameter management

Web Interface:

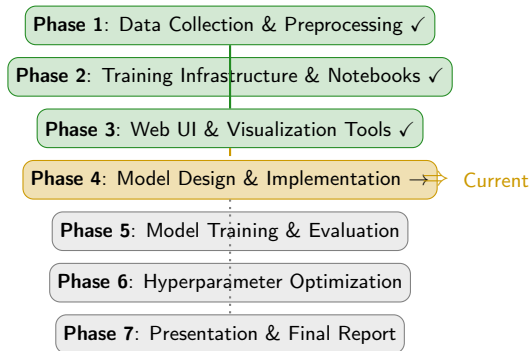
- **Streamlit** — Interactive UI
- Real-time ECG visualization
- Model loading & inference
- Anomaly detection dashboard

Infrastructure:

- **Git** — Version control
- **LaTeX** — Documentation
- **pytest** — Unit testing
- Modular package structure



Completed Work Summary



Detailed Progress — Completed

Data Pipeline:

- ✓ MIT-BIH NSRDB & MITDB datasets loaded
- ✓ Resampling, bandpass filtering, segmentation
- ✓ Quality control pipeline
- ✓ Subject-wise data splitting
- ✓ PyTorch Dataset/DataLoader

Training Infrastructure:

- ✓ Jupyter notebooks for training
- ✓ YAML configuration system
- ✓ MLflow experiment tracking
- ✓ Multiple config presets

Web Interface & Visualization:

- ✓ Streamlit application
- ✓ Model loading & inference
- ✓ ECG signal visualization
- ✓ QRS detection module
- ✓ Feature extraction
- ✓ Comprehensive report generation

Experiment Configurations

Table: Pre-defined Training Configurations

Parameter	Baseline	Fast	High Cap.	Best
Epochs	50	20	100	100
Batch Size	32	64	16	32
Learning Rate	0.001	0.002	0.0005	0.001
Dropout	0.2	0.1	0.3	0.2
Kernel Size	7	7	7	7
Latent Channels	1024	512	2048	1024

Configs are stored as YAML files in `experiments/configs/`
and can be run via: `python experiments/train.py -config <config.yaml>`

Immediate Next Steps

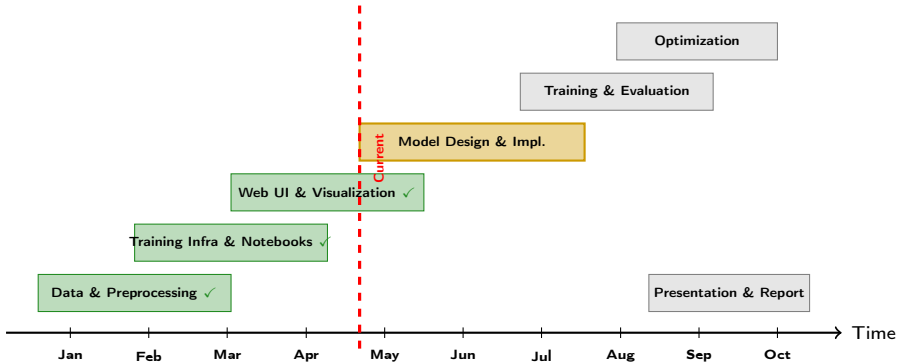
Phase 4: Model Design & Implementation (— Current —)

- Design autoencoder architecture
- Implement encoder-decoder layers
- Define loss functions (reconstruction-based)
- Set up training loop & validation
- Model checkpointing & versioning

Phase 5: Training & Evaluation

- Train autoencoder on normal ECG data
- Monitor training/validation loss curves
- Evaluate reconstruction quality
- Test on MIT-BIH Arrhythmia Database
- Compute metrics: AUC, F1, Precision, Recall

Project Timeline



Summary of Achievements

What I Have Done

- ❶ **Complete data pipeline** — From raw PhysioNet signals to PyTorch-ready tensors
- ❷ **Training infrastructure** — CLI, notebooks, config system, MLflow tracking
- ❸ **Anomaly detection framework** — Reconstruction error thresholding
- ❹ **Interactive web UI** — Streamlit app for real-time visualization & detection

Current Focus — Phase 4

Data preprocessing, infrastructure, and web UI are complete. The current focus is on **designing the autoencoder architecture** and **implementing the model**, which will lay the groundwork for training and evaluation.

Thank You!

Questions & Discussion

A.K.S.S. Wimalasena

Department of Mathematics

University of Ruhuna

Supervised by **Ms. Sachini Karunarathne**

April 2026